

Classification of Malicious and Benign Memory Signatures Using Memory Forensics and Machine Learning

^aSonika Malik, Assistant Professor

^bVedaang Chopra, Student

Department of IT, MSIT, Delhi, India

Abstract

Malwares have been a constant threat to every individual and organization. Even though Antivirus and End Point Security tool exists still, the number of attacks and damage cost keeps increasing exponentially. Systems are compromised and hefty amounts need to be paid by organizations to release their data. Everyday a private user is a victim of such Malware Attacks. The use of signature-based mapping to malwares can prevent only known malwares but zero-day malwares and obfuscated malwares can easily bypass, this technique. The need for an AI based approach is the need of hour and major antivirus providers can be seen shifting to such techniques. The technique that has been implemented in this research is analysing malware based on their memory values. It combines the concept of Memory Forensics, YARA Rules and Artificial Intelligence.

Keywords: Artificial Intelligence, Endpoint Security, Memory Forensics, YARA Rules

1. Introduction

End Point Security mechanic system type of security mechanism which deals with protection of a range of devices including laptop, desktop, mobile etc. This specifically works on protecting the device only from all the attacks, which could be local or on a network.eg. Some endpoint security solutions for personal devices include antiviruses, firewalls, etc. [1]. Malware, or “malicious software,” is an umbrella term that describes any malicious program or code that is harmful to systems [2][3].

Some common types of malware are [4]:

- A virus is malware that attaches to another program and, when executed—usually inadvertently by the user—replicates itself by modifying other computer programs and infecting them with its own bits of code.
- Worms are a type of malware similar to viruses, self-replicating in order to spread to other computers over a network, usually causing harm by destroying data and files.
- A Trojan, or Trojan horse, is one of the most dangerous malware types. It usually represents itself as something useful in order to trick you. Once it's on your system, the attackers behind the Trojan gain unauthorized access to the affected computer. From

there, Trojans can be used to steal financial information or install threats like viruses and ransomware.

- Ransomware is a form of malware that locks you out of your device and/or encrypts your files, then forces you to pay a ransom to get them back. Ransomware has been called the cyber criminal's weapon of choice because it demands a quick, profitable payment in hard-to-trace cryptocurrency. The code behind ransomware is easy to obtain through online criminal marketplaces and defending against it is very difficult.
- Rootkit is a form of malware that provides the attacker with administrator privileges on the infected system. Typically, it is also designed to stay hidden from the user, other software on the system, and the operating system itself.
- A key logger is malware that records all the user's keystrokes on the keyboard, typically storing the gathered information and sending it to the attacker, who is seeking sensitive information like usernames, passwords, or credit card details.
- Exploits are a type of malware that takes advantage of bugs and vulnerabilities in a system in order to allow the exploit's creator to take control. Among other threats, exploits are linked to malicious advertising, which attacks through a legitimate site that unknowingly pulls in malicious content from a bad site. Then the bad content tries to install itself on your computer in a drive-by download. No clicking is necessary. All you have to do is visit a good site on the wrong day.

Antiviruses have deployed various techniques to classify a file as malicious or not [2][3] [5]:

- a. Signature based Detection: In this detection technique the traditional method of matching any program to its signature (Example: hash) is employed. It is a premature technique and can be easily bypassed as a slight change in the program can immediately change its signature. For this technique to work seamlessly it requires a large database of known malicious files.
- b. Heuristic based Detection: This is the most common form of detection that uses an algorithm to compare the signature of known viruses against a potential threat. An antivirus packed with this type of detection can also detect viruses that have not yet been discovered and released as a new virus but it can also generate false positive match

which means that an antivirus scanner may report an uninfected file as an infected one.

- c. Behavioral-based detection: Using the dynamic malware analysis technique this detection method monitors the malicious program and after analyzing the changes made by the process to the system, it classifies whether the file is a malware or not. Some of the characteristics of the process which are monitored are as follows:
- Creation of files
 - Deletion of files
 - Memory consumed

Sandbox detection: This is a type of detection method in which antivirus software run programs in a virtual environment and record the actions it performs to identify whether the programs are malicious or not. If the program is found safe, it is then executed in the real environment.

2. Background Information

The following concepts are needed to understand this research [6][7]:

2.1. Malware Analysis

The task of analyzing malwares which is identifying what unique signatures a malware has is known as Malware Analysis. These signatures can include strings in the malware code, author name, server to which it connects etc. It is categorized into two parts [8]:

Static Analysis

This is a process where to try analysing those properties of a malware which can be identified without executing it. Some examples of such properties are:

- i. File type
- ii. PE Header
- iii. Packer used
- iv. Strings etc....

Dynamic Analysis

This is a process where to try analysing those properties of a malware which can be identified by executing it in an environment. Some examples of such properties are:

- v. Resource consumption
- vi. Network traffic generated
- vii. Files Dropped/Created etc.

a. Memory Forensics

Digital forensics, a branch of forensic science is works on the principle of encompassing the recovery and investigation of material found in digital devices, often in relation to computer crime [9].

Memory Forensics is a field of digital forensics that involves recovery and investigation of a digital devices' memory [10].

Volatility: - It is a tool used to perform Memory Forensics. The Volatility Framework is a completely open collection of tools, implemented in Python under the GNU General Public License, for the extraction of digital artefacts from volatile memory (RAM) samples.

The extraction techniques are performed completely independent of the system being investigated but offer visibility into the runtime state of the system. The framework is intended to introduce people to the Techniques and complexities associated with extracting digital artefacts from volatile memory samples and provide a platform for further work into this exciting area of research [11].

2.2. YARA

YARA is a tool that identifies malware by creating descriptions that look for certain characteristics. Each description can be either a text or a binary pattern [12]. These descriptions are called "rules". And by using rules that specify regex patterns, YARA enables the detection of specific patterns in files that might indicate that the file is malicious.

By using hex patterns, plain text patterns, wild-cards, case-insensitive strings, and special operators, YARA rules can be incredibly diverse and effective at detecting a wide range of malware signatures [13].

2.3. Machine Learning

It is a methodology in which we train our machines to learn from our past experiences [14].

Example: Human beings learn crossing a road with experience. They analyze the road and make the decision correspondingly.

Machine learning is classified into three types:

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

Supervised Learning: It is a type of learning that has a clear cut output marked for all specific inputs.

It is of two types: Regression and Classification. We have solved a classification problem using supervised learning in our research. The classifiers used are as follows [15].

Naive Bayes classifier-

This algorithm is good for multi-class classification as it uses Bayes theorem to predict values.

Consider a data point x, in a multi-class scenario with three classes- A, B and C. Using Naïve Bayes, we try to predict whether the data point x belongs to class A or B or C, by calculating its probability for the three classes.

$$P(A/B) = (P(B/A) \cdot P(A))/P(B)$$

(1)

This algorithm is called 'Naïve', because it assumes that all the features are independent of each other.

$$P(f_1, f_2, f_3, \dots, f_n) = P(f_1) = P(f_2) = \dots = P(f_n) \quad (2)$$

There are further two categories of the NB Classifier, the Gaussian Naïve Bayes Classifier, and the multinomial Naïve-Bayes Classifier.

The Gaussian Naïve-Bayes classifier is used when a dataset has continuous values of data. It uses the Gaussian Probability Distribution function (values are centred around mean and as the graph grows, the values decrease).

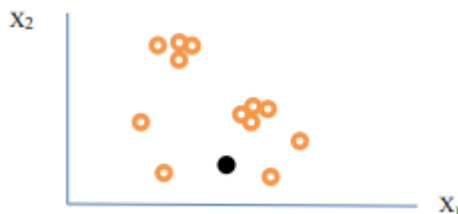
Decision Tree-

An ensemble technique which deals with creating a tree based on decisions formulated from the dataset. To construct a decision tree, we have to perform the following steps – (i) select a feature to split the data, (ii) select a method to split the data on the said feature. It has the internal working algorithm as:

- (i) Create/Select a node.
- (ii) If the node is pure, output the only class
- (iii) If no feature is left to split upon, and the node is impure, output the majority class.
- (iv) Else find the best feature to split upon. Recursively call on this split. Goto step (ii)

KNN

Consider a scenario where we have to predict to which class, the testing point belongs to, by considering all the features at once. Such is the working of KNN algorithm as shown in Fig 1. To predict the class of the testing data point, we check its vicinity.



To classify the testing point, we check a specific number of points (1, 3, 5, 7, etc.) and whichever class is in majority

Fig. 1 KNN Algorithm

Gradient Boosting Algorithm

It continuously creates models that are nothing but improvisation on the previous version. It essentially keeps strengthening the model until a model strong enough for correct prediction is generated. This is done by building a model from the training data, then creating a second model that attempts to correct the errors from the first model. The addition of improved models terminates when either of the two outcomes are achieved – (i) the prediction is perfected or (ii) the upper limit for adding models is reached.

3. Related Work

among those, that one is predicted. To select the nearest point, we have to consider its distance from the other points. The distance metric can be (a) Manhattan distance, (b) Euclidian Distance, (c) Makowski distance.

Random forest

It is an extension of the decision tree classifier. This algorithm uses multiple combinations of decision trees to accurately predict testing data. The random forest classifier overcomes the overfitting problem of decision trees by building multiple decision trees and going with the majority result. The trees' outputs vary because each tree is built with random data and random features. To generate randomness in trees, we use two techniques-

Bagging Classifier: If we have 'm' data points, we select a subset of 'k' out of them. For 'n' trees, n*k subsets are selected. Data points can be considered with replacement as the selection is random, therefore, these trees are called bag trees.

Feature Selection: In the training phase, some features are selected at random in this technique, with the condition that the selection is performed without replacement.

SVM Classifier

It is a very powerful algorithm and overcomes the limitation of logistic regression. As logistic regression uses sigmoid function, the value predicted for a testing data point is close to 0.5. This causes the problem of incorrect prediction. So, SVM uses the rules of logistic regression only, but exponentially increases the value, so that the values predicted do not fall in the range (-1, 1). The cost function of the logistic regression is:

$$E(h(x)) = \sum (-y_i \log(h(x)) - (1-y_i) \log(1-h(x))) \quad (3)$$

This cost function changes to the following equation in SVM –

$$(\theta) = C \sum [y_{(i)} \text{cost}1(\theta T_{x(i)}) + (1-y_{(i)} \text{cost}0(\theta T_{x(i)}))] + 0.5 \sum (\theta_i)^2 \quad (4)$$

Various techniques have been used by researchers to analyse and detect malware. In order to detect malware, several researchers used static analysis. Machine learning was used by Hashemi and Hamzeh [16] to detect malware after analysing executable files to extract the unique opcode features. According to Salehi et al. [17], the most frequent API calls were selected to learn the classifier. Cheng et al. [18] extracted native APIs from shellcode using WinDbg and classified the malware using SVM. Based on static analysis, Sun et al. [19] identified malicious PE files with 32-bit and 64-bit opcode sequences. In order to distinguish different types of malicious PE files, the frequency of each opcode sequence is calculated. Data sets from Kaggle Microsoft Malware Classification Challenge (BIG 2015) and vx-heaven were analysed. By applying the

Adaboost classifier, the highest accuracy equalled 98.57%. A gradient-based attack by Kolosnjaji et al. [20] was also developed to evade malware detection by modifying a few bytes at the end of each malware file. Furthermore, dynamic analysis has been used by many authors in the detection and classification of malware. An in-depth evaluation of the techniques and tools that have been used in dynamic analysis has been published by Egele et al. in [21]. Zeus malware was classified by Mohaisen et al. [22] using the registry file system and network features extracted from the registry. This led to the introduction of a behaviour-based automated malware analysis and labelling system (AMAL) by Mohaisen et al. [23]. File system, registry, and network activity were studied to identify malware. A technique introduced by Liang et al. in [24] uses API dependency chains to calculate the similarity between malware variants. Galal et al. [25] similarly captured API call information and its parameters. Sequences were created for API calls with mutual purposes. The API hooking approach was also used by Fan et al. [26] in order to trace hidden API calls as well as native APIs. A similar sequence was aligned using a multiple

sequence alignment algorithm (MSA) by Ki et al. [27]. According to Dai et al. [28], a memory dump can be used to categorize malware. An image in gray scale is created by extracting files from the memory dump. A fixed size is then applied to the image. A feature extraction algorithm was applied to the images to categorize malware based on API call features. A random forest was used to achieve the highest accuracy of 93.9%. In spite of the successful results achieved by the above methods, no other work attempts to integrate dynamic analysis with memory analysis to overcome their limitations.

4. Proposed Framework

The Project flow can be classified into 4 separate phases as shown in Fig. 2.

1. Lab Setup
2. Sample Generation
3. Memory Analysis

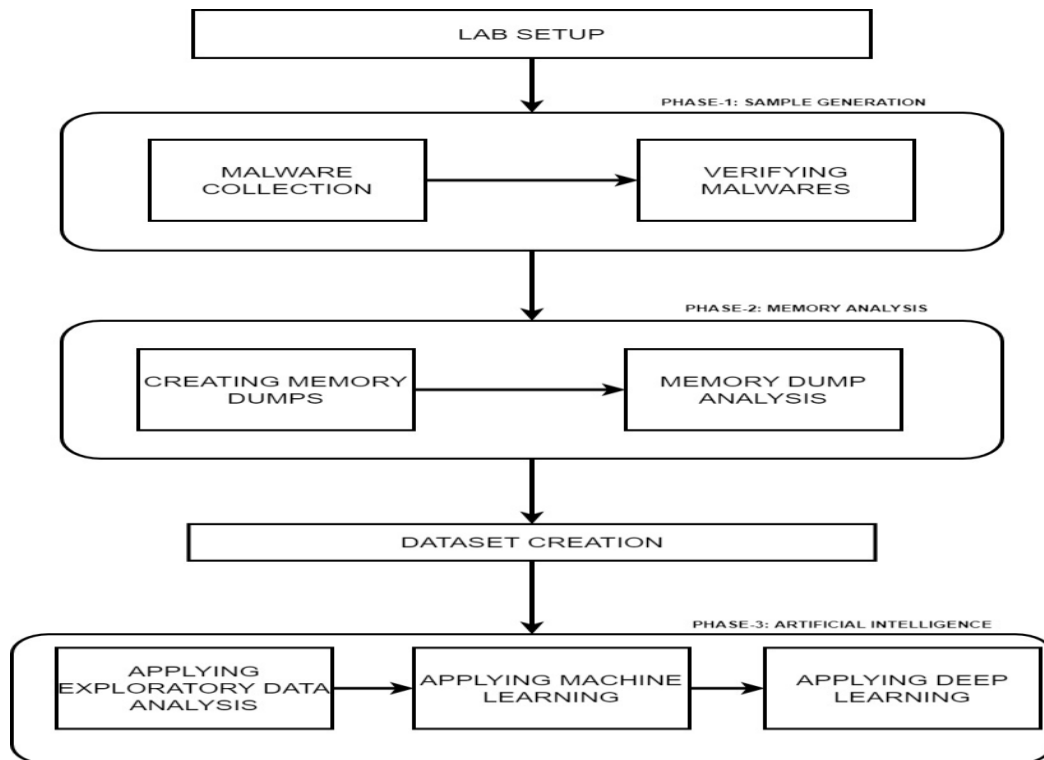


Fig. 2. Flow of Code of Project

4.1. Lab Setup

To analyse malware, there is a need to setup a complete LAB including hypervisors, malware analysis tools, forensics tools etc.

The following Labs were created: -

1. Malware Analysis Lab Setup:

- a. VMware machine with Windows 10 (trial)
- b. Installation of FLARE-VM over Windows OS

2. Memory Forensics Lab Setup:

- a. VMware machine with UBUNTU 18.04
- b. Installation of CUCKOO Sandbox
- c. Installation of Virtual box Ubuntu Server with Inetsim
- d. Installation of Virtual box machine with Windows 10 inside VM machine

4.2. Sample Generation

To collect live malwares, many public repositories were used. Some of the repositories were:-

1. Any.apprun
2. theZOO
3. <https://github.com/Endermanch/MalwareData> base
4. malware-samples@fabrimagic

Even though the samples were collected, still each of the sample needed to be verified by running it in a sandboxed environment. For initial analysis and dataset creation process only RANSOMWARES were considered.

To verify each malware each was executed within a virtual machine, with guest operating system as Windows 10 x64, Build version 18362.(Windows Developer trial)

The list of Ransomwares used are as follow:

1. Annabelle.exe
2. Blue Howl.exe
3. Ceber.exe
4. Cerber1.exe(another version)
5. GandCrab.exe
6. Jigsaw1.exe
7. Jigsaw2.exe(another version)
8. TeslaCrypt1.exe
9. Vipasana1.exe
10. ViraLock.exe
11. WannaCry1.exe

12. WannaCry2.exe(another version)

4.3. Memory Analysis

This Memory Analysis phase consists of two parts:

- a. Creating Memory Dumps
- b. Analyzing Memory Dumps

Creating Memory Dumps:

To create a memory dump, of a malware executed sandbox these techniques are used:

- Use of Cuckoo Sandbox Automated Memory Dump feature.
- Use of VirtualBox command line feature.

COMMAND TO USE

```
VBoxManage debugvm <Virtual Machine name> dumpvcore -- filename=<filename>.elf
```

Analyzing Memory Dumps:

To analyze memory dumps use of Volatility (memory forensics tool) is required. It is used to extract all crucial information about the systems. In our project, the following steps and commands were used to analyze memory [16][17].

Steps:

1. Create the .elf into .raw file for proper analysis using the imagecopy command. (elf files have issue. See http://wiki.yobi.be/wiki/RAM_analysis for further details.)
2. Identify the build profile of the memory dump using imageinfo command.
3. Identify all the processes running in the system using the pslist command.
4. Identify all the hidden processes by running the psscan command and comparing the results with pslist command output.
5. With your Yara rules collection run the yarascan command to identify malicious memory samples.
6. Use this result for analysis. Repeat the following steps for other memory dumps.

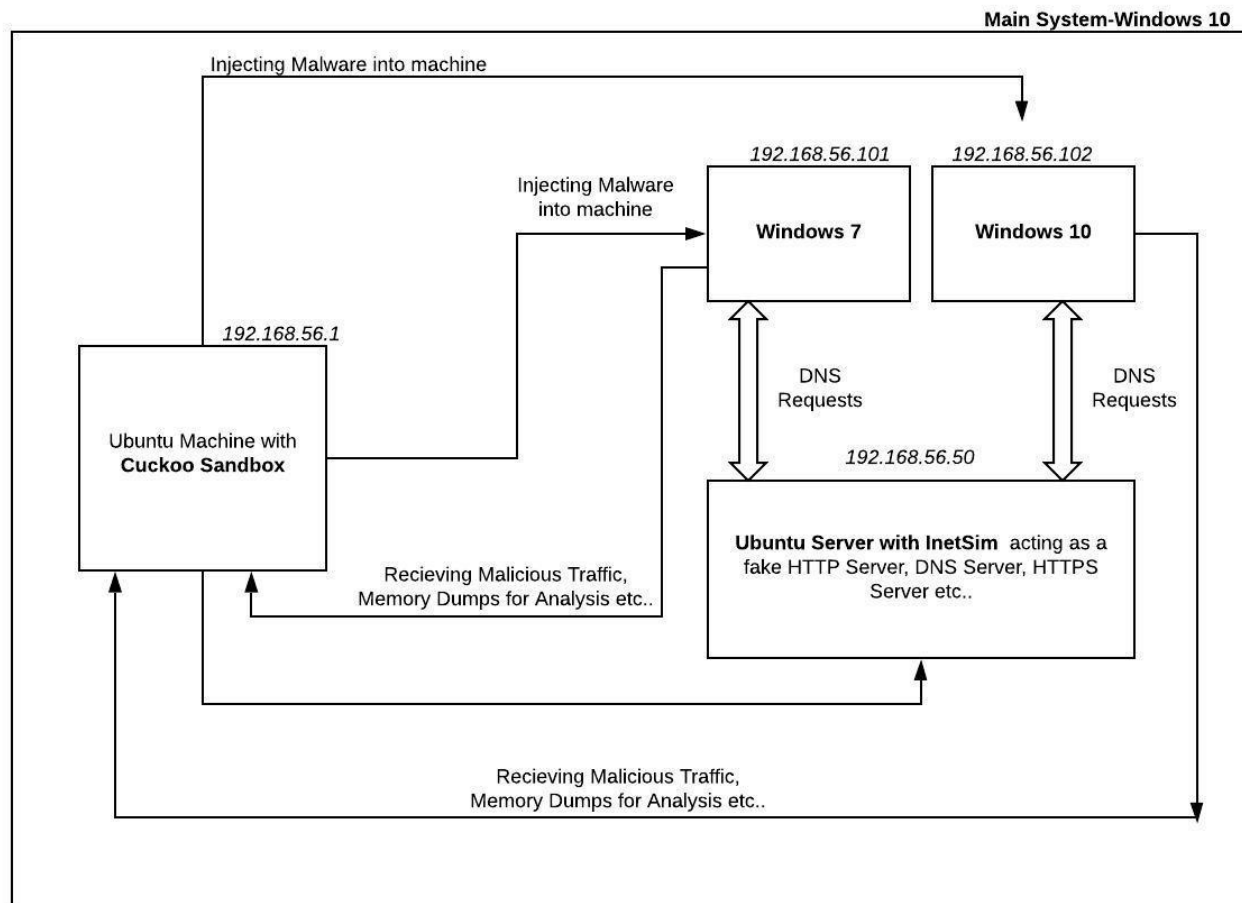
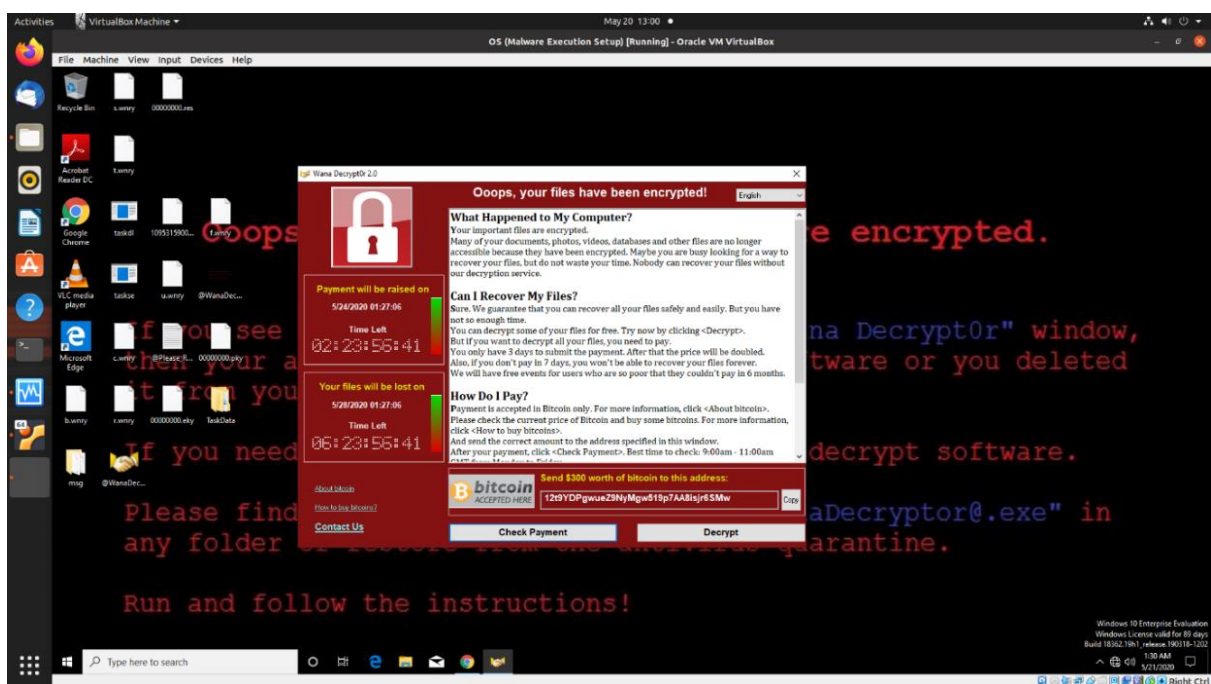


Fig. 3. Memory Forensics Lab Setup




```

cuckoo@ubuntu:/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis$ vol.py -f WannaCry2.raw imageinfo
Volatility Foundation Volatility Framework 2.6.1
INFO : volatility.debug : Determining profile based on KDBG search...
Suggested Profile(s) : Win10x64_18362
AS Layer1 : SkipDuplicatesAMD64PagedMemory (Kernel AS)
AS Layer2 : FileAddressSpace (/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis/WannaCry2.raw)
PAE type : No PAE
DTB : 0x1aa002L
KDBG : 0xf8053b8265e0L
Number of Processors : 4
Image Type (Service Pack) : 0
KPCR for CPU 0 : 0xfffff8053855b000L
KPCR for CPU 1 : 0xfffff8180b5fc0000L
KPCR for CPU 2 : 0xfffff8180b6021000L
KPCR for CPU 3 : 0xfffff8180b5be3000L
KUSER_SHARED_DATA : 0xfffff78000000000L
Image date and time : 2020-05-10 09:31:13 UTC+0000
Image local date and time : 2020-05-10 15:01:13 +0530
cuckoo@ubuntu:/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis$

```

Fig. 6. Identifying Build profile of the Memory Dump.

```

Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x00879f28 74 61 73 6b 73 65 2e 65 78 65 00 00 40 57 61 6e taskse.exe..@Wan
0x00879f38 61 44 65 63 72 79 70 74 6f 72 40 2e 65 78 65 00 aDecryptor@.exe.
0x00879f48 74 61 73 6b 73 63 68 65 2e 65 78 65 00 00 00 tasksche.exe...
0x00879f58 25 00 73 00 5c 00 25 00 73 00 5c 00 25 00 73 00 %s.\.%s.\.%s.
0x00879f68 00 00 00 00 25 00 73 00 5c 00 2a 00 2e 00 2a 00 ...%.s.\.*...*.
0x00879f78 00 00 00 40 57 61 6e 61 44 65 63 72 79 70 74 ...@WanaDecrypt
0x00879f88 6f 72 40 2e 65 78 65 2e 6c 6e 6b 00 5c 00 00 00 or.exe.lnk.\...
0x00879f98 40 65 63 68 6f 20 6f 66 66 0d 0a 65 63 68 6f 20 @echo.off..echo.
0x00879fa8 53 45 54 20 6f 77 20 3d 20 57 53 63 72 69 70 74 SET.ow.=.WScript
0x00879fb8 2e 43 72 65 61 74 65 4f 62 6a 65 63 74 28 22 57 .CreateObject("W
0x00879fc8 53 63 72 69 70 74 2e 53 68 65 6c 6c 22 29 3e 20 Script.Shell")>.
0x00879fd8 6d 2e 76 62 73 0d 0a 65 63 68 6f 20 53 45 54 20 m.vbs..echo.SET.
0x00879fe8 6f 6d 20 3d 20 6f 77 2e 43 72 65 61 74 65 53 68 om.=.ow.CreateSh
0x00879ff8 6f 72 74 63 75 74 28 22 25 73 25 73 22 29 3e 3e ortcut("%s%s")>>
0x0087a008 20 6d 2e 76 62 73 0d 0a 65 63 68 6f 20 6f 6d 2e .m.vbs..echo.om.
0x0087a018 54 61 72 67 65 74 50 61 74 68 20 3d 20 22 25 73 TargetPath.="s
Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x0087a0a8 72 2e 77 6e 72 79 00 00 62 2e 77 6e 72 79 00 00 r.wnry..b.wnry..
0x0087a0b8 61 74 74 72 69 62 20 2b 68 20 2b 73 20 25 43 3a attrib.+h.+s.%C:
0x0087a0c8 5c 25 73 00 24 52 45 43 59 43 4c 45 00 00 00 %s.$RECYCLE...
0x0087a0d8 25 00 43 00 3a 00 5c 00 25 00 73 00 00 00 00 %C:..\.%s...
0x0087a0e8 24 00 52 00 45 00 43 00 59 00 43 00 4c 00 45 00 $.R.E.C.Y.C.L.E.
0x0087a0f8 00 00 00 00 25 00 73 00 5c 00 68 00 69 00 62 00 ...%.s.\.h.i.b.
0x0087a108 73 00 79 00 73 00 25 00 73 00 00 00 20 00 3a 00 s.y.s.%s.....
0x0087a118 5c 00 00 00 74 61 73 6b 64 6c 2e 65 78 65 00 00 \...taskdl.exe..
0x0087a128 25 73 0a 00 66 2e 77 6e 72 79 00 00 61 74 00 00 %s..f.wnry..at..
0x0087a138 63 6d 64 2e 65 78 65 20 2f 63 20 73 74 61 72 74 cmd.exe./c.start
0x0087a148 20 2f 62 20 25 73 20 76 73 00 00 25 73 20 63 o...taskkill.exe
0x0087a158 6f 00 00 00 74 61 73 6b 6b 69 6c 6c 2e 65 78 65 ./f./im.mysqld.e
0x0087a168 20 2f 66 20 2f 69 6d 20 6d 79 73 71 6c 64 2e 65 xe..taskkill.exe
0x0087a178 78 65 00 00 74 61 73 6b 6b 69 6c 6c 2e 65 78 65 ./f./im.sqlwrite
0x0087a188 20 2f 66 20 2f 69 6d 20 73 71 6c 77 72 69 74 65 r.exe...taskkill
0x0087a198 72 2e 65 78 65 00 00 74 61 73 6b 6b 69 6c 6c
Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x0087a074 75 2e 77 6e 72 79 00 00 25 2e 31 66 20 42 54 43 u.wnry..%.1f.BTC
0x0087a084 00 00 00 00 24 25 64 20 77 6f 72 74 68 20 6f 66 ...$.%d.worth.of
0x0087a094 20 62 69 74 63 6f 69 6e 00 00 00 77 00 62 00 .bitcoin....w.b.
0x0087a0a4 00 00 00 00 72 2e 77 6e 72 79 00 00 62 2e 77 6e ...r.wnry..b.wn
0x0087a0b4 72 79 00 00 61 74 74 72 69 62 20 2b 68 20 2b 73 ry..attrib.+h.+s
0x0087a0c4 20 25 43 3a 5c 25 73 00 24 52 45 43 59 43 4c 45 %C:\%s.$RECYCLE
0x0087a0d4 00 00 00 00 25 00 43 00 3a 00 5c 00 25 00 73 00 ...%.C:..\.%s.
0x0087a0e4 00 00 00 00 24 00 52 00 45 00 43 00 59 00 43 00 ...$.R.E.C.Y.C.
0x0087a0f4 6c 00 45 00 00 00 00 00 25 00 73 00 5c 00 68 00 l.E....%.s.\.h.
0x0087a104 49 00 62 00 73 00 79 00 73 00 25 00 73 00 00 00 i.b.s.y.s.%s...
0x0087a114 20 00 3a 00 5c 00 00 00 74 61 73 6b 64 6c 2e 65 .../\...taskdl.e
0x0087a124 78 65 00 00 25 73 0a 00 66 2e 77 6e 72 79 00 00 xe..%.s..f.wnry..
0x0087a134 61 74 00 00 63 6d 64 2e 65 78 65 20 2f 63 20 73 at..cmd.exe./c.s
0x0087a144 74 61 72 74 20 2f 62 20 25 73 20 76 73 00 00 00 tart./b.%s.vs...
0x0087a154 25 73 20 63 6f 00 00 74 61 73 6b 6b 69 6c 6c %s.co...taskkill
0x0087a164 2e 65 78 65 20 2f 66 20 2f 69 6d 20 6d 79 73 71 .exe./f./im.mysq

```

Fig. 7. Yarascan Result using Volatility

4.4. Yara Rules

The yara rules are collection of signatures that have been analyzed by malware analysts, that are used to identify malwares. These rules are certain conditions specific for each malware and help to identify it. Use of these rules

have been done by majorly every endpoint security company, such as Virustotal, mcafee, palo Alto etc [15].

4.5. Creating a Dataset

After creating scans and storing the results, the need is to model these scan results into a dataset. To model these

results into dataset, use the following flowchart as shown in Figure 8.

Steps:-

1. Create Yarascan results of each malware using volatility
2. Remove common signatures that it has by matching it with *standard benign samples*
3. Append these rules into a dataframe.
4. Keep record of benign and malicious samples by a creating a **TARGET** column

4.6. Using Artificial Intelligence to Predict Malware

After the creation of Dataset, it was time to create the dataset. This involved steps such as:

a. Data Pre-Processing

- a. Merging data from different Scan Results
- b. Handling all Duplicated Values
- c. Scaling Data: - Dividing data values by 255 (16*16 memory samples produced) to scale data between 0 and 1.
- d. Splitting data into Training and Testing

b. Exploratory Data Analysis

The Heat map doesn't produce much result. It depicts how features between the range (13,104) show slight correlation as shown in Figure 9. This could help to reduce dimensions.

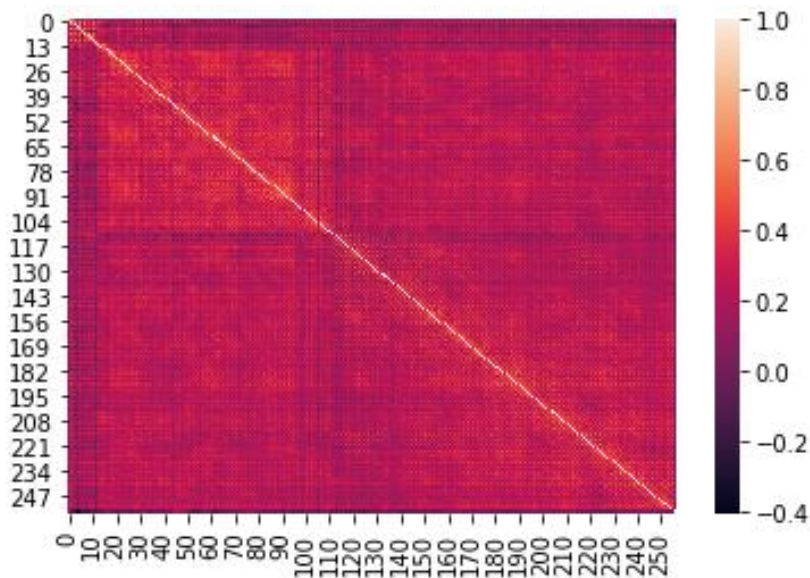


Fig. 9. Heat Map showing data correlation between features

The Count plot of Target Column shows the number of samples of each class shown in Figure 10. On Applying PCA and TSNE to reduce dimensions to visualize each

the samples, it can be conclusively said that samples tend to form compact clusters as shown in Figure 11 and 12.

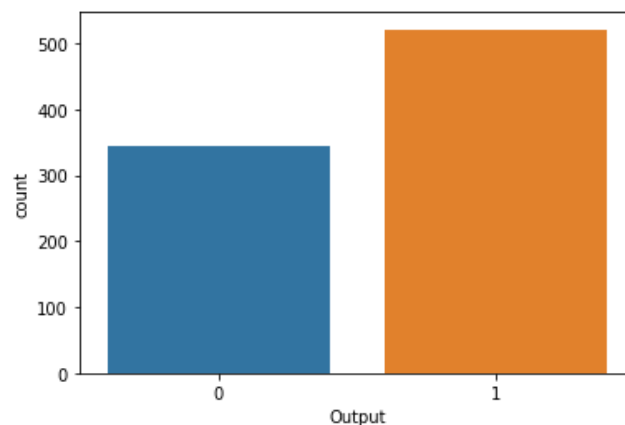


Fig. 10. Distribution of target column

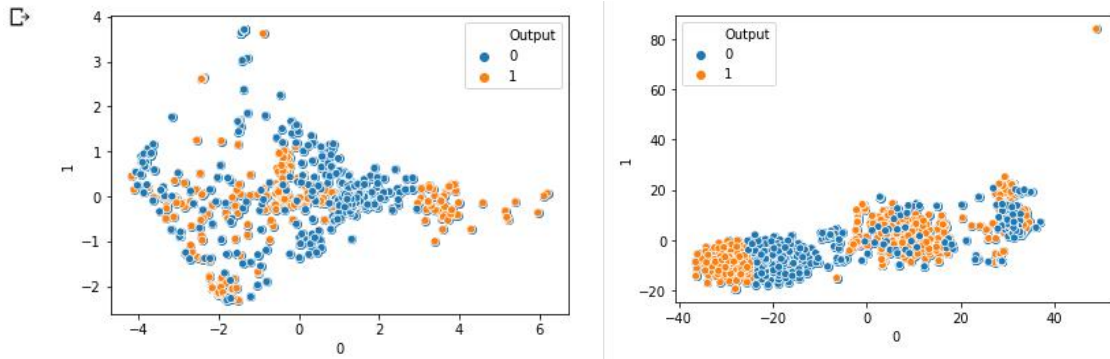


Fig. 11. Reduced Information Distribution of Malware and Benign Samples

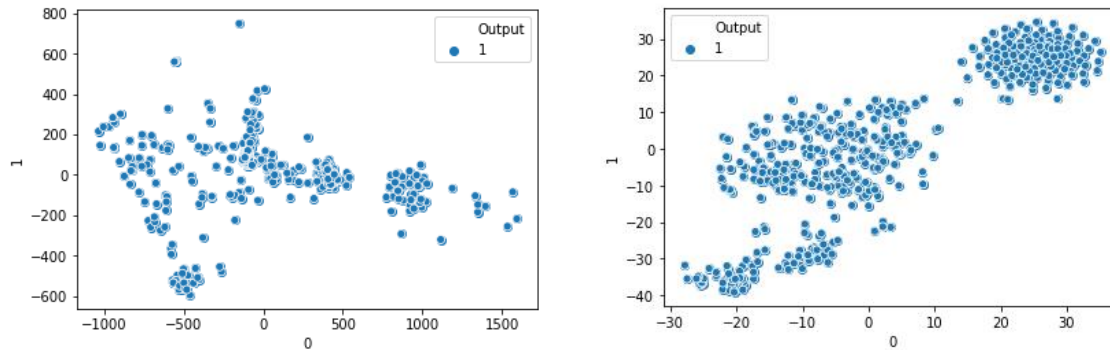


Fig. 12. Reduced Information Distribution of Malware samples only

This helps to identify that classification of such compact clusters, needs a highly refined and complex decision boundary/ function.

- c. Applying Machine Learning Algorithms and Deep learning Algorithms
- On Applying various Machine Learning Algorithms, the metrics accuracy, precision, recall and F1-Score were computed which are reported. The detailed discussion on results are discussed in the next chapter.
 - Classical supervised learning algorithms seem to perform poorly.
 - Ensemble techniques seem to perform better on the dataset.

- High Training Score is obtained
- The Deep learning algorithm used currently is ANN (Artificial Neural Network). It consists of dense and dropout layers. The model needs a better model majorly a model that can remember the previous value, such as an RNN or LSTM etc.
- The following example of KNN algorithm proves are PCA and TSNE graphs/plots as shown in Figure 13. As the clusters were compact and in nature, and malware and benign samples were close to each other, the value of K had to be low. The score/accuracy should decrease with increase in K-nearest neighbours.

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                    metric_params=None, n_jobs=None, n_neighbors=3, p=2,
                    weights='uniform')
Score of KNN on Training Data: 0.7682119205298014
Score of KNN on Testing Score: 0.6923076923076923
```

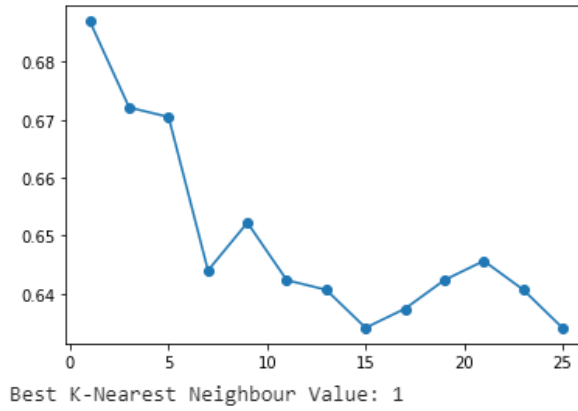


Fig. 13. Using KNN with dataset and mapping score with K using cross validation

5. Results and Discussion

From the Machine Learning Analysis, we could clearly see that clusters were being formed by malicious samples and benign samples. Thus processes exhibiting similar type of

behaviour in system have similar memory values in memory. Using this when we applied machine learning and deep learning to dataset we got the following result as shown in Table 1.

Table 1 Values of different parameters for all classifiers after classification

Name of Classifier	Accuracy	Precision	Recall	F1 Score
KNN Classifier	0.692	0.71	0.69	0.70
SVM Classifier	0.769	0.77	0.77	0.77
Naïve Bayes Classifier	0.657	0.66	0.66	0.66
Logistic Regression	0.730	0.73	0.73	0.73
Decision Tree Classifier	0.80	0.81	0.81	0.81
Random Forest Classifier	0.857	0.86	0.86	0.86
Gradient Boosting Classifier	0.85	0.85	0.85	0.85
Bagging Classifier	0.857	0.86	0.86	0.86
AdaBoost Classifier	0.792	0.79	0.79	0.79

On Applying Machine Learning, on the memory values the model seems to perform well as shown in Fig. 14. It has an average training score and is able to predict the new malwares from the previously learnt malwares. The dataset needs to be increased for it to have a higher accuracy.

On Applying Machine Learning, on the memory values the model seems to perform well as shown in Fig. 15. It has a high training score because which it should as it needs to remember all the values of the malwares given to it.

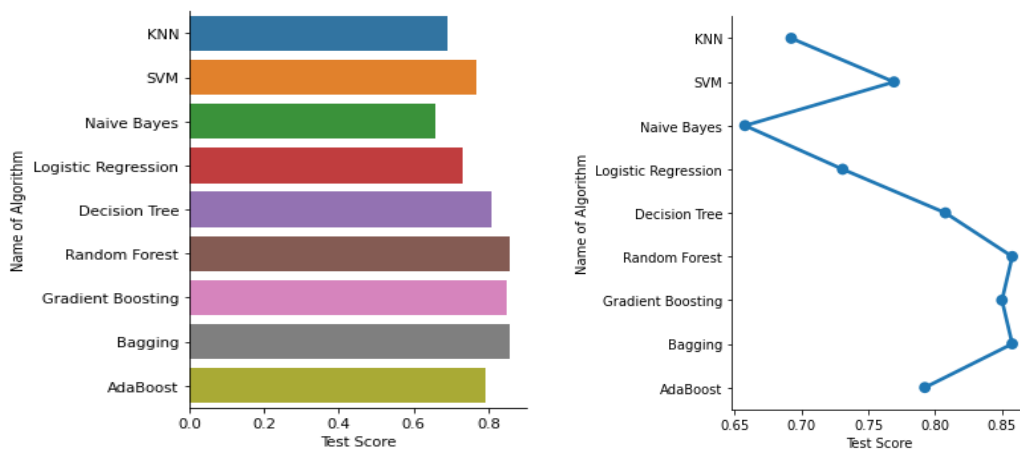


Fig. 14. Comparing Testing Scores of Dataset

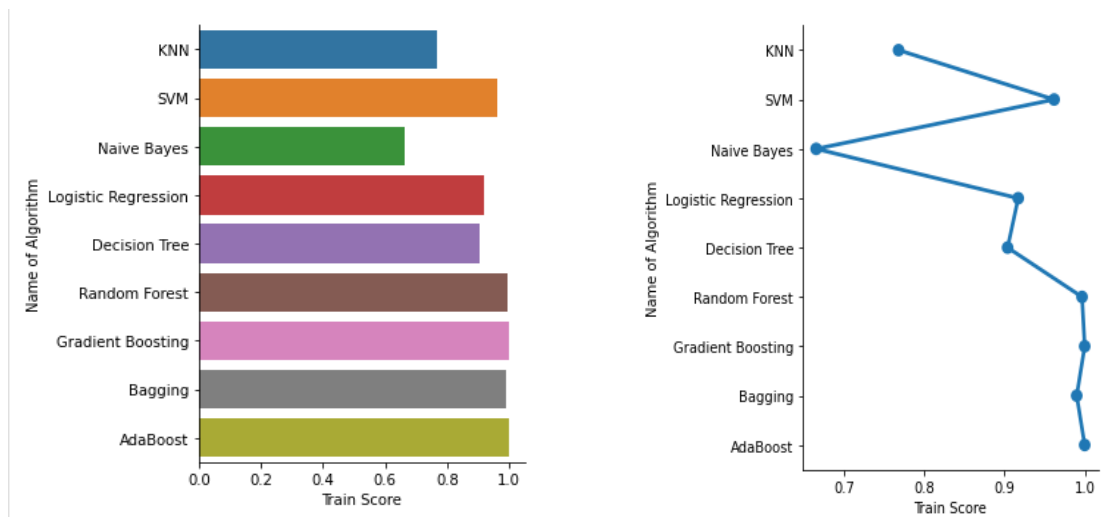


Fig. 15. Comparing Training Scores of Dataset

6. Conclusion and Future Scope

A method for classifying malicious files on the end-user computer was developed in this study that is highly effective and reliable. This study aims to improve malware detection accuracy and reduce false positives. Experiments have proven that features extracted from dynamic analysis are more accurate when more related features are added from memory. By looking for extra features in memory that don't appear in dynamic analysis, the proposed approach overcomes the limitation of one path of execution in dynamic analysis, thereby revealing an unnoticed part of malware and improving the performance of the proposed detection approach.

For Future scope, the list of YARA rules can be expanded to include other open source rules from Alienvault, McAfee, Palo Alto etc. The technique if it is to be utilized as a full scale antivirus needs an interface and can be upgraded if the use of rules is eliminated and in depth analysis of malwares is done and how they affect the memory hexadecimal values.

References

- [1] <https://medium.com/bugbountywriteup/intro-to-malware-detection-using-yara-eacab8373cf4>
- [2] Ye, Y., Li, T., Adjeroh, D., & Iyengar, S. S. (2017). A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)*, 50(3), 1-40.

- [3] Sihwail, R., Omar, K., & Ariffin, K. Z. (2018). A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis. *Int. J. Adv. Sci. Eng. Inf. Technol*, 8(4-2), 1662-1671.
- [4] <https://github.com/volatilityfoundation/volatility>
- [5] <https://www.geeksforgeeks.org/how-an-antivirus-works/>
- [6] Bhojani, N. (2014). Malware Analysis. *Malware Analysis*, 1-5.
- [7] Tahir, R. (2018). A study on malware and malware detection techniques. *International Journal of Education and Management Engineering*, 8(2), 20.
- [8] <https://www.malwarebytes.com/malware/>
- [9] <https://antivirus.comodo.com/how-antivirus-software-works.php>
- [10] https://www.blackhat.com/presentations/bh-dc-07/Kendall_McMillan/Presentation/bh-dc-07-Kendall_McMillan.pdf
- [11] <https://yara.readthedocs.io/en/v3.8.1/index.html>
- [12] Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, L., & Brown, D. (2019). Text classification algorithms: A survey. *Information*, 10(4), 150.
- [13] <https://yara.readthedocs.io/en/v3.8.1/index.html>
- [14] https://en.wikipedia.org/wiki/Digital_forensics
- [15] <https://resources.infosecinstitute.com/category/computerforensics/introduction/areas-of-study/digital-forensics/memory-forensics/#gref>
- [16] Hashemi, H.; Hamzeh, A. Visual malware detection using local malicious pattern. *J. Comput. Virol. Hacking Tech*. 2018, 15, 1–14.
- [17] Salehi, Z.; Sami, A.; Ghiasi, M. Using feature generation from API calls for malware detection. *Comput. Fraud Secur*. 2014, 2014, 9–18.
- [18] Cheng, Y.; Fan, W.; Huang, W.; An, J. A Shellcode Detection Method Based on Full Native API Sequence and Support Vector Machine. In *IOP Conference Series: Materials Science and Engineering*; IOP Publishing: Bristol, UK, 2017; Volume 242, pp. 1–7.
- [19] Sun, Z.; Rao, Z.; Chen, J.; Xu, R.; He, D.; Yang, H.; Liu, J. An Opcode Sequences Analysis Method For Unknown Malware Detection. In *Proceedings of the 2019 2nd International Conference on Geoinformatics and Data Analysis*, Prague, Czech Republic, 15–17 March 2019; ACM: New York, NY, USA, 2019.
- [20] Kolosnjaji, B.; Demontis, A.; Biggio, B.; Maiorca, D.; Giacinto, G.; Eckert, C.; Roli, F. Adversarial malware binaries: Evading deep learning for malware detection in executables. In *Proceedings of the 2018 26th European Signal Processing Conference (EUSIPCO)*, Rome, Italy, 3–7 September 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 533–537.
- [21] Egele, M.; Scholte, T.; Kirda, E.; Kruegel, C. A survey on automated dynamic malware-analysis techniques and tools. *ACM Comput. Surv*. 2012, 44, 6.
- [22] Mohaisen, A.; Alrawi, O. Unveiling Zeus Automated Classification of Malware Samples. In *Proceedings of the 22nd International Conference on World Wide Web Companion*, Rio de Janeiro, Brazil, 13–17 May 2013; pp. 829–832.
- [23] Mohaisen, A.; Alrawi, O.; Mohaisen, M. AMAL: High-fidelity, behavior-based automated malware analysis and classification. *Comput. Secur*. 2015, 52, 251–266.
- [24] Liang, G.; Pang, J.; Dai, C. A Behavior-Based Malware Variant Classification Technique. *Int. J. Inf. Educ. Technol*. 2016, 6, 291–295.
- [25] Galal, H.S.; Mahdy, Y.B.; Atiea, M.A. Behavior-based features model for malware detection. *J. Comput. Virol. Hacking Tech*. 2016, 12, 59–67.
- [26] Fan, C.-I.; Hsiao, H.-W.; Chou, C.-H.; Tseng, Y.-F. Malware Detection Systems Based on API Log Data Mining. In *Proceedings of the 2015 IEEE 39th Annual Computer Software and Applications Conference*, Los Alamitos, CA, USA, 10 June 2015; pp. 255–260.
- [27] Ki, Y.; Kim, E.; Kim, H.K. A novel approach to detect malware based on API call sequence analysis. *Int. J. Distrib. Sens. Netw*. 2015, 11, 659101.
- [28] Dai, Y.; Li, H.; Qian, Y.; Lu, X. A malware classification method based on memory dump grayscale image. *Digit. Investig*. 2018, 27, 30–37.